BASE DE DATOS II — GUÍAS DE LABORATORIO

Semana 04: Gestión de Esquemas e Índices en Azure SQL Database

PaaS Compatible

Base de datos: Grados_Y_Titulos · Guías de laboratorio N° 01 al 05

GUÍA N° 01: CONSOLIDACIÓN Y VERIFICACIÓN DE ESQUEMAS LÓGICOS

1. OBJETIVO

Consolidar y verificar esquemas lógicos funcionales en la base de datos para agrupar las tablas por módulo de negocio, evitando el uso exclusivo del esquema por defecto `dbo` y facilitando la gestión de permisos por rol (RBAC) con el principio de mínimo privilegio.

2. FUNDAMENTO TEÓRICO

Los esquemas lógicos permiten agrupar los objetos de una base de datos (como tablas y vistas) según su módulo o área funcional de negocio (por ejemplo: **Catalogo, Academico, Tramite, Evaluacion, Documento** y **Seguridad**).

Su uso evita la mala práctica de concentrar todo bajo el esquema genérico `dbo`, facilitando la aplicación de controles de acceso corporativo basados en roles (RBAC) y asegurando que cada usuario o rol posea únicamente los permisos estrictamente necesarios.

3. PROCEDIMIENTO PRÁCTICO

T-SQL

```
-- Creación de esquemas funcionales
IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = 'Catalogo')
    EXEC('CREATE SCHEMA Catalogo;');
IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = 'Academico')
    EXEC('CREATE SCHEMA Academico;');
IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = 'Tramite')
    EXEC('CREATE SCHEMA Tramite;');
IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = 'Evaluacion')
    EXEC('CREATE SCHEMA Evaluacion;');
IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = 'Documento')
    EXEC('CREATE SCHEMA Documento;');
IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = 'Seguridad')
    EXEC('CREATE SCHEMA Seguridad;');
GO

-- Verificación
SELECT schema_id, name AS NombreEsquema
FROM sys.schemas
WHERE name IN ('Catalogo', 'Academico', 'Tramite',
               'Evaluacion', 'Documento', 'Seguridad');
GO
```

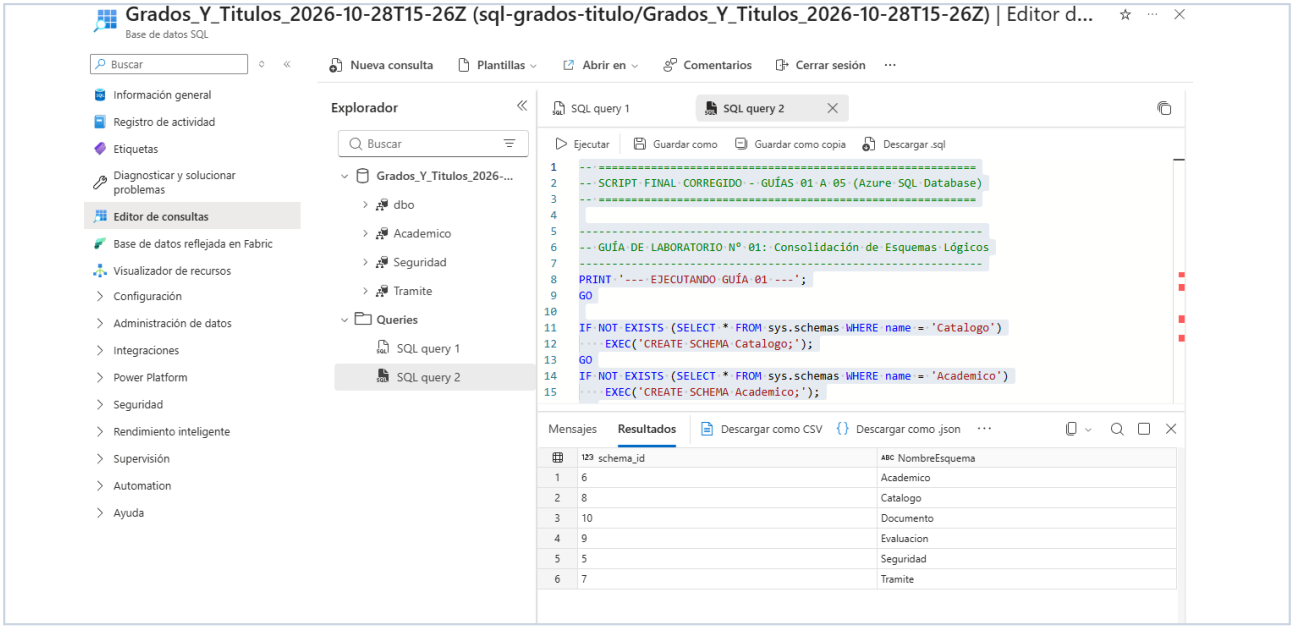


Figura 1. Ejecución de la Guía 01 en el Editor de consultas de Azure SQL Database: los seis esquemas funcionales creados.

4. PREGUNTA DE VALIDACIÓN

¿De qué manera la separación de entidades en esquemas lógicos mejora la aplicación del principio de mínimo privilegio en el control de acceso corporativo (RBAC)?

Respuesta: La separación de entidades en esquemas lógicos mejora la aplicación del principio de mínimo privilegio porque permite segmentar las tablas y objetos por módulos funcionales de negocio específicos. Gracias a esto, los administradores pueden otorgar permisos de manera granular y estricta a cada rol corporativo (por ejemplo, permitir que un área solo acceda al esquema `Tramite`), impidiendo que los usuarios tengan acceso innecesario al resto de la base de datos.

GUÍA N° 02: IMPLEMENTACIÓN DE ÍNDICES CLÚSTER ESTRATÉGICOS

1. OBJETIVO

Implementar y auditar índices clúster estratégicos en las tablas principales de la base de datos, reconociendo que este tipo de índice define el orden físico de almacenamiento de las filas y ayuda a prevenir la fragmentación ante inserciones masivas mediante una clave adecuada.

2. FUNDAMENTO TEÓRICO

El índice clúster define el orden físico en que se guardan las filas de una tabla dentro del motor de base de datos.

Debido a que la información física solo puede ordenarse de una sola forma, **solo puede existir un índice clúster por tabla**. Elegir una clave adecuada y autoincremental previene de manera directa la fragmentación durante las inserciones masivas de datos.

3. PROCEDIMIENTO PRÁCTICO

T-SQL

```
-- Auditoría de índices clúster existentes
SELECT
    s.name      AS Esquema,
    t.name      AS Tabla,
    i.name      AS IndiceClustered,
    i.type_desc AS Tipo
FROM sys.indexes i
INNER JOIN sys.tables  t ON i.object_id = t.object_id
INNER JOIN sys.schemas s ON t.schema_id = s.schema_id
WHERE i.type = 1    -- 1 indica CLUSTERED
ORDER BY s.name, t.name;
GO
```

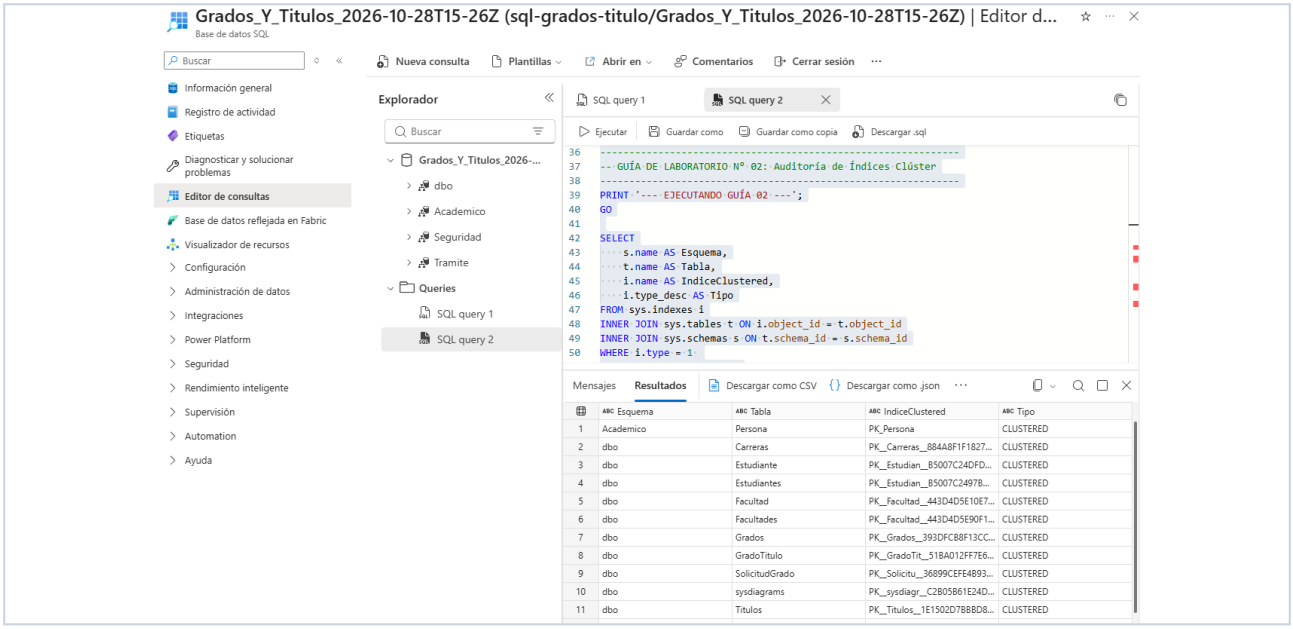


Figura 2. Resultado de la consulta de auditoría: tablas principales con su índice clúster (CLUSTERED).

4. PREGUNTA DE VALIDACIÓN

¿Por qué es fundamental que cada tabla principal posea únicamente un índice clúster y de qué manera la elección de una clave adecuada previene la fragmentación en inserciones masivas?

Respuesta: Es fundamental porque el índice clúster determina directamente el orden físico en el que se almacenan las filas de la tabla en el disco, por lo que solo puede existir uno por cada tabla. Asimismo, elegir una clave adecuada y secuencial previene la fragmentación en inserciones masivas porque asegura que los nuevos registros se coloquen de forma ordenada al final de la estructura, evitando costosas divisiones de páginas (*page splits*).

GUÍA N° 03: CREACIÓN DE ÍNDICES NO CLÚSTER CON COLUMNAS INCLUIDAS

1. OBJETIVO

Crear índices no clúster incorporando columnas adicionales mediante la cláusula `INCLUDE`, generando índices cubiertos que eviten operaciones costosas de búsqueda (*Key Lookups*) en las consultas frecuentes.

2. FUNDAMENTO TEÓRICO

Los índices no clúster almacenan columnas clave y punteros que apuntan hacia los datos principales.

Al utilizar la cláusula `INCLUDE`, se añaden columnas extra directamente en las hojas del índice (sin formar parte de la clave de búsqueda), creando así **índices cubiertos**. Esto permite que el motor de base de datos resuelva la consulta directamente desde el índice sin necesidad de realizar costosos accesos adicionales a las páginas de datos de la tabla (*Key Lookups*).

3. PROCEDIMIENTO PRÁCTICO

T-SQL

```
-- Índices cubiertos con la cláusula INCLUDE
IF NOT EXISTS (SELECT * FROM sys.indexes WHERE name = 'IX_Persona_DNI_Cover'
               AND object_id = OBJECT_ID('Academico.Persona'))
BEGIN
    CREATE NONCLUSTERED INDEX IX_Persona_DNI_Cover
    ON Academico.Persona (DNI)
    INCLUDE (Nombres, ApellidoPaterno, ApellidoMaterno, Correo, TipoPersona);
END;
GO

IF NOT EXISTS (SELECT * FROM sys.indexes WHERE name = 'IX_Persona_TipoPersona'
               AND object_id = OBJECT_ID('Academico.Persona'))
BEGIN
    CREATE NONCLUSTERED INDEX IX_Persona_TipoPersona
    ON Academico.Persona (TipoPersona)
    INCLUDE (DNI, Nombres, ApellidoPaterno, ApellidoMaterno);
END;
GO
```

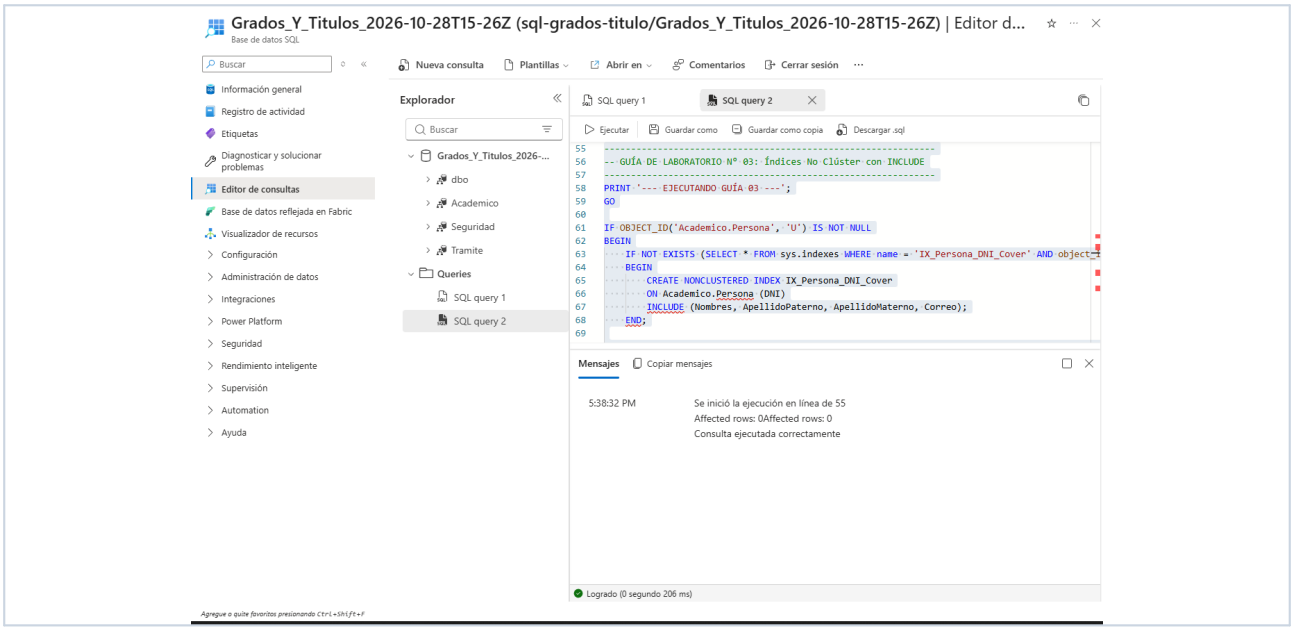


Figura 3. Ejecución correcta del script de índices no clúster sobre la tabla Academico.Persona.

4. PREGUNTA DE VALIDACIÓN

¿De qué manera la cláusula INCLUDE en los índices no clúster optimiza el rendimiento al evitar operaciones de *Key Lookup*?

Respuesta: La cláusula INCLUDE optimiza el rendimiento porque permite almacenar columnas adicionales en los niveles hoja del índice sin agregarlas a la clave de búsqueda principal. De este modo, si una consulta solicita columnas que están incluidas dentro del índice, SQL Server puede obtener toda la información requerida directamente de la estructura del índice, eliminando la necesidad de buscar en las páginas de datos de la tabla base (*Key Lookup*) y acelerando drásticamente la respuesta.

GUÍA N° 04: INDEXACIÓN COMPUESTA PARA CONSULTAS ANALÍTICAS DE TRÁMITES

1. OBJETIVO

Implementar índices compuestos considerando críticamente el orden de sus columnas (primero igualdad y luego rango) para optimizar las consultas analíticas y los reportes ejecutados por el optimizador de SQL Server.

2. FUNDAMENTO TEÓRICO

En los índices compuestos, el orden de las columnas es un factor crítico para su rendimiento.

Se debe colocar en primer lugar la columna de **igualdad** (por ejemplo, el estado) y posteriormente la columna de **rango** (por ejemplo, la fecha). Gracias a esta jerarquía estructural, el optimizador de SQL Server puede aprovechar el índice de manera sumamente eficiente para procesar reportes y consultas complejas sin incurrir en escaneos de tabla completos.

3. PROCEDIMIENTO PRÁCTICO

T-SQL

```
-- Índices compuestos: primero igualdad, luego rango
IF NOT EXISTS (SELECT * FROM sys.indexes WHERE name = 'IX_Tramite_Estado_Fecha'
               AND object_id = OBJECT_ID('Tramite.Tramite'))
BEGIN
    CREATE NONCLUSTERED INDEX IX_Tramite_Estado_Fecha
    ON Tramite.Tramite (IdEstadoTramite, FechaInicio)
    INCLUDE (IdMatricula, IdTipoTramite, IdCoordinacion);
END;
GO

IF NOT EXISTS (SELECT * FROM sys.indexes WHERE name = 'IX_Pago_Tramite_Fecha'
               AND object_id = OBJECT_ID('Tramite.Pago'))
BEGIN
    CREATE NONCLUSTERED INDEX IX_Pago_Tramite_Fecha
    ON Tramite.Pago (IdTramite, FechaPago)
    INCLUDE (Monto, NroRecibo, TipoPago);
END;
GO
```

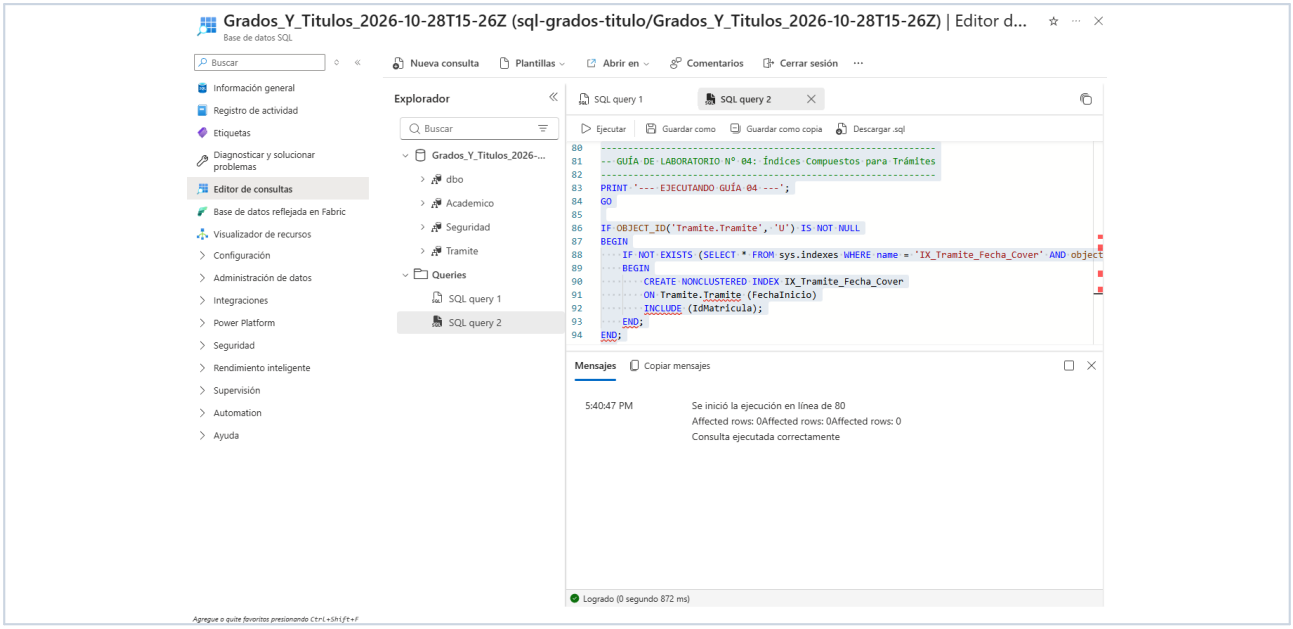


Figura 4. Ejecución correcta del script de índices compuestos para trámites y pagos.

4. PREGUNTA DE VALIDACIÓN

¿Por qué es crítico el orden de las columnas en los índices compuestos (como IX_Tramite_Estado_Fecha) y cómo interviene el optimizador de consultas de SQL Server en los reportes analíticos?

Respuesta: El orden de las columnas es crítico porque el árbol B-Tree del índice organiza físicamente la información respetando estrictamente la jerarquía definida al crearlo (primero las columnas de igualdad y luego las de rango). Esto permite que el optimizador de consultas de SQL Server navegue directamente hacia los datos solicitados en los reportes analíticos de manera óptima, maximizando la velocidad de respuesta y evitando operaciones de lectura innecesarias.

GUÍA N° 05: AUDITORÍA, MONITOREO Y ANÁLISIS DE FRAGMENTACIÓN DE ÍNDICES

1. OBJETIVO

Auditar, monitorear y analizar el estado de fragmentación física de los índices en la base de datos para tomar decisiones técnicas fundamentadas entre las operaciones de mantenimiento como `REORGANIZE` y `REBUILD`.

2. FUNDAMENTO TEÓRICO

Debido a las operaciones constantes de inserción, actualización y borrado (DML), los índices de la base de datos sufren de fragmentación física con el paso del tiempo.

Monitorear periódicamente su estado mediante vistas de administración dinámica permite evaluar el porcentaje de fragmentación para aplicar la acción correctiva adecuada:

Fragmentación (%)	Acción recomendada
Menor a 5%	No requiere mantenimiento.
Entre 5% y 30%	**REORGANIZE**: reorganización ligera de las páginas hoja.
Mayor a 30%	**REBUILD**: reconstrucción completa del índice.

3. PROCEDIMIENTO PRÁCTICO

T-SQL

```
-- 1. Auditoría general de todos los índices creados por esquema y tabla
SELECT
    s.name            AS Esquema,
    t.name            AS Tabla,
    i.name            AS NombreIndice,
    i.type_desc       AS TipoIndice,
    i.is_primary_key  AS EsClavePrimaria,
    i.is_unique       AS EsUnico
FROM sys.indexes i
INNER JOIN sys.tables  t ON i.object_id = t.object_id
INNER JOIN sys.schemas s ON t.schema_id = s.schema_id
WHERE i.name IS NOT NULL
ORDER BY s.name, t.name, i.type;
GO

-- 2. Análisis de fragmentación física de los índices
SELECT
    OBJECT_SCHEMA_NAME(ps.object_id)  AS Esquema,
    OBJECT_NAME(ps.object_id)         AS Tabla,
    i.name                            AS Indice,
    ps.avg_fragmentation_in_percent   AS PorcentajeFragmentacion,
    ps.page_count                     AS CantidadPaginas
FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'LIMITED') ps
INNER JOIN sys.indexes i
    ON ps.object_id = i.object_id AND ps.index_id = i.index_id
WHERE ps.avg_fragmentation_in_percent > 10
ORDER BY ps.avg_fragmentation_in_percent DESC;
GO
```

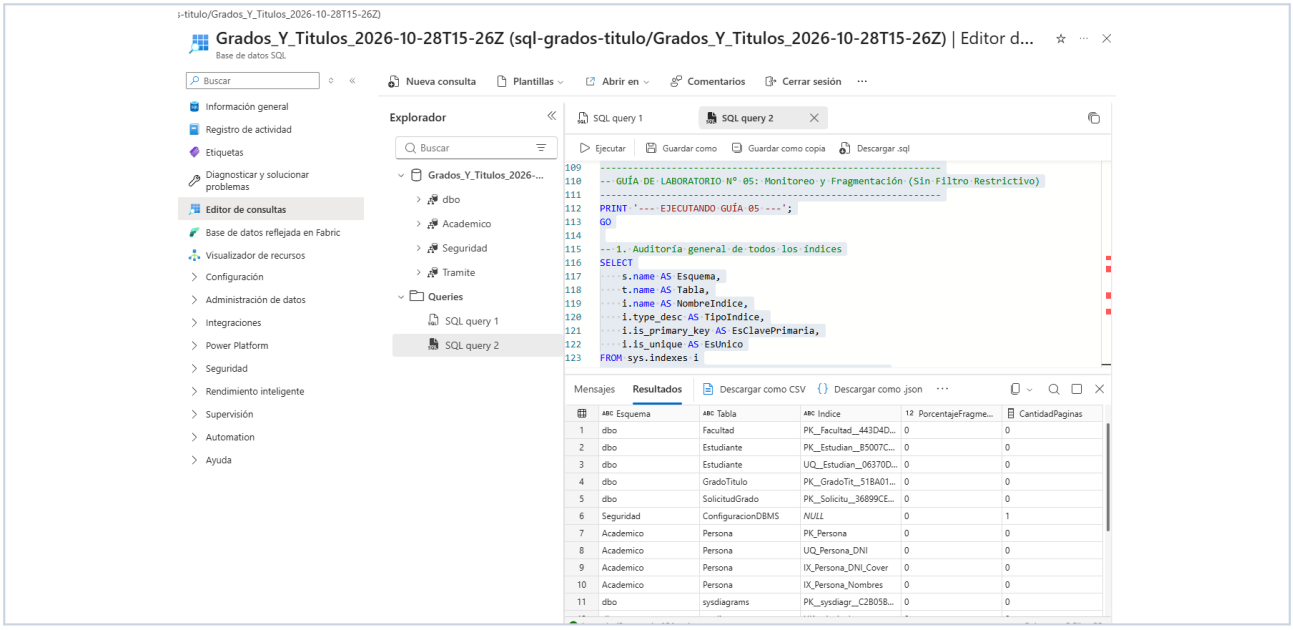


Figura 5. Resultado del análisis de fragmentación con sys.dm_db_index_physical_stats.

4. PREGUNTA DE VALIDACIÓN

¿Qué criterios se deben considerar al analizar los resultados de fragmentación física para decidir entre aplicar un mantenimiento de tipo REORGANIZE o un REBUILD sobre un índice?

Respuesta: Los criterios se basan estrictamente en el porcentaje de fragmentación física (avg_fragmentation_in_percent) obtenido a través de las vistas dinámicas del sistema. Se debe aplicar REORGANIZE cuando la fragmentación se encuentra en un rango moderado (entre 5% y 30%), ya que realiza una reorganización ligera de las páginas de nivel hoja de forma eficiente. Por el contrario, se debe aplicar un REBUILD (reconstrucción) cuando el porcentaje de fragmentación supera el 30%, debido a que reestructura por completo el índice de manera profunda para recuperar óptimamente el rendimiento de lectura y escritura.